
Overcooked

Alessio Arcara
University of Bologna
alessio.arcara@studio.unibo.it

Abstract

The abstract paragraph should be indented 1/2 inch (3 picas) on both the left- and right-hand margins. Use 10 point type, with a vertical spacing (leading) of 11 points. The word **Abstract** must be centered, bold, and in point size 12. Two line spaces precede the abstract. The abstract must be limited to one paragraph.

1 Introduction

Overcooked-AI[2] environment was designed a where the goal is to place three onions in a pot, take the resulting soup on a plate and delivering it

that allows benchmark and evaluate reinforcement learning (RL) agents on their ability to cooperate with unknown partners in unfamiliarity environments

This work chooses **Independent Proximal Policy Optimization (IPPO)** as the learning algorithm for the cooperative multi-agent reinforcement learning (MARL) problem. Under independent learning, a MARL problem is decomposed into n interdependent learning problems, which are solved independently by each agent. Each agent treats the other agents as part of the environment and learns its own policy and value function based solely on local observations.

While this approach is simple and scalable, it violates the stationarity assumption required by Markov Decision Processes (MDPs), since the environment dynamics change as other agents update their policies. As a result, independent learning may suffer from non-stationarity.

Centralized training methods attempt to address non-stationarity by using global information during training, either through joint policy learning or centralized value functions. However, recent empirical results show that IPPO can match or even outperform CTDE methods, such as QMIX and MAPPO [3, 6].

The clipped surrogate objective of PPO effectively mitigates non-stationarity issues, making IPPO a simple and effective baseline for cooperative MARL, without requiring a centralized critic conditioned on the global state or on the concatenation of agents observations, as in MAPPO.

2 Background

2.1 Proximal Policy Approximation

2.2 Independent PPO

3 Methodology

3.1 Implementation

3.2 Reward Shaping

The reward signal in Overcooked is extremely sparse: the environment assigns a reward of 20 only when a soup is successfully delivered. However, a successful delivery is a complex interaction requiring a long sequence of coordinated actions. Specifically, an agent must collect three ingredients, move to the pot, place the ingredients, start cooking, pick up a dish, return to the pot, collect the soup, move to the delivery area, and finally deliver it. This must be accomplished while avoiding invalid recipes, which result in zero reward.

Because this chain of operations is so long, reinforcement learning algorithms struggle to figure out which intermediate steps actually contributed to the final success. This significantly slows down learning.

By looking rendered frames and the environment code, we found that Overcooked already implements a reward shaping mechanism, returned at each step through the *info* dictionary as a dense shaped reward (`ep_shaped_r`). The environment assigns fixed positive values for specific events such as ingredient placement in the pot (3), dish pickup (3), and soup pickup (5).

Although this reward-shaping scheme is relatively simple, it proved to be highly effective. The use of a dense shaped reward enabled the learning algorithm to quickly develop an understanding of how to prepare soups. We did observe that this form of reward shaping can lead to *reward hacking*; for example, early in training the agents repeatedly picked up and dropped dishes to exploit the reward signal. However, these behaviors were transient. As training progressed, the agents moved beyond such exploits and converged to stable, effective policies that exhibited correct behavior.

Since these shaped rewards are generated per agent, we designed a Gymnasium wrapper with two distinct modes to handle them:

- **Shared Mode:** The dense rewards obtained by all agents are summed and distributed equally. This means both agents get credit whenever a beneficial action happens, regardless of who did it.
- **Individual Mode:** The dense reward is assigned only to the agent that performed the action.

In our experiments, shared mode achieved faster convergence and higher final total reward. Consequently, it can be interpreted as a team-based reward that promotes coordinated learning.

3.3 State Representation

By default, the Overcooked environment returns a **featurized state** observation, represented as a fixed-length vector of size 96 designed to reduce state complexity. This hand-engineered encoding replaces the raw grid with the agent’s absolute position, orientation, and relative distances to key entities such as the co-player, ingredients, and pots [1].

In contrast, the **lossless state** encoding preserves the complete spatial topology of the environment. It is structured as a tensor of shape $W \times H \times 26$, where W and H represent the layout dimensions. The 26 channels serve as semantic layers, with each channel acting as a binary mask indicating the presence of specific objects at given coordinates. To investigate the trade-off between engineered features and complete state information, we modified the environment to selectively return either the featurized state or the raw lossless tensor [1].

3.4 Architectures

For featurized states, we employed a three-layer MLP with $\tanh$ activations, consistent with the standard PPO2 implementation [4].

For lossless states, we used a modified Nature-CNN architecture [5] restricted to kernel sizes of 3×3 with a stride of 1. Given the small spatial dimensions of the layouts, larger kernels or strides would excessively downsample the feature maps, collapsing the grid into a single cell and destroying local spatial context.

All hidden layers and convolutions used *orthogonal initialization* (scale $\sqrt{2}$, bias 0). The policy head weights were initialized with a scale of 0.01 to ensure a uniform initial action distribution at start for maximum exploration, while the value head used a scale of 1.0 to immediately match the magnitude of expected returns [4].

We also explored sharing parts of the network between the policy and the value function, either by sharing two MLP layers or by using a CNN backbone with separate final policy and value heads. While this approach effectively improves computational efficiency, since the shared layers are computed fewer times,

In addition, we also explored the possibility of sharing some layers between the policy and the value function. In particular, we experimented with sharing either two MLP layers or a CNN backbone, followed by a single final layer acting as a policy head and a value head. Although this approach effectively improves computational efficiency—since the shared two or three layers are computed only once, it negatively affects convergence and degrades the final return achieved. This behavior is consistent with previous observations, as the separate-networks architecture clearly outperforms the shared one in simpler environments. The inferior performance of the shared-network architecture is likely due to the competing objectives of the policy and value functions, as discussed in [4].

3.5 Experiments



Figure 1: Training layouts: Cramped Room, Asymmetric Advantage, Coordination Ring, Counter Circuit, and Forced Coordination.

To understand the model capacity to generalize, we use a wrapper of the Overcooked-AI environment that samples one layout from a list of layouts at every reset. In this way, we hope the agent pair becomes invariant to the layout and understands how to generalize.

We select 5 layouts for the **training set**: *Cramped Room*, *Asymmetric Advantage*, *Coordination Ring*, *Counter Circuit*, and *Forced Coordination* (Figure 1). We evaluate generalization on a separate **test set**: *Cramped Room Tomato* (introduces tomatoes) and a 5×5 (more open-space).

Since every layout has different dimensions in width and height, this presents a problem for lossless encoding. Because the Nature-CNN inspired architecture involves a flattening step after the convolutional layers, and the subsequent fully connected layer requires fixed dimensions, it is necessary to pad each layout to the maximum dimension among all layouts.

Every 50 batches during training, the agent is evaluated in **self-play** on all 5 training layouts. Training runs for 10M environment steps resulting in roughly 400 batches. Each layout is tested with 5 rollouts, and the total reward across layouts is summed. The agent pair achieving the highest total reward is saved. At the end of training, the best agent pair is reloaded and evaluated again on all layouts, including the holdout set, using 10 rollouts per layout.

In addition to the generalized agent pair, we also trained agent pairs on each individual training layout. This let us see the maximum reward an agent can achieve when it focuses solely on one

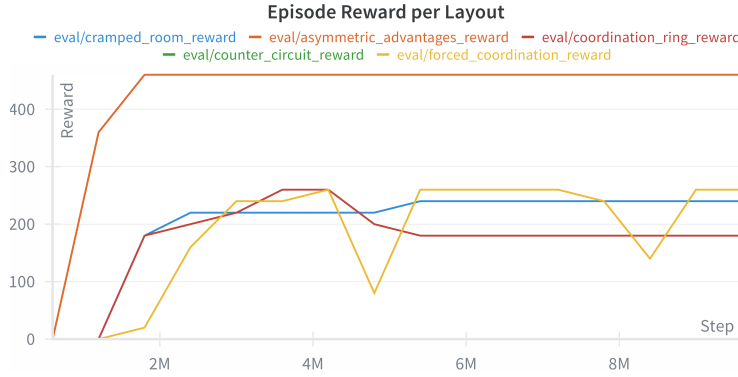


Figure 2: Episode reward over training steps for each layout, using the multi-layout agent pair trained with the best hyperparameters.

layout, and gave us a baseline to compare against the generalized pair. By looking at this comparison, we can understand what the generalized agent learns across multiple layouts, and whether trying to be versatile comes at the cost of performing optimally on any single layout.

3.5.1 Hardware & Software

Experiments were conducted on a desktop machine equipped with a 16-core CPU, 32 GB of RAM, and an NVIDIA GeForce RTX 3090 GPU with 24GB of VRAM, used for both rollout collection and training updates. We developed the implementation using **Jax** and **Flax**, which helped significantly with computation speed. It is important to note that a Jax implementation of the environment, such as the one available in JaxMARL, would allow for much faster rollout collection. This phase was the main bottleneck in our pipeline, as the GPU often remained underutilized while waiting for the CPU to finish collecting rollouts. Although we avoided switching to a Jax-native environment to align with the specific requirements of the project, such a transition would be incredibly beneficial to maximize computational throughput.

Furthermore, since the primary bottleneck is rollout collection, a **vectorized environment** was utilized to run multiple environments in parallel across separate processes using Gymnasium AsyncVectorizeEnv. In this setting, the agent pair interacts simultaneously across N environments, significantly increasing training throughput. The number of environments was set to approximately the number of available CPU cores. It is worth noting that a native Jax implementation would allow N to be increased by orders of magnitude.

By leveraging this setup, we achieved a training time of approximately 30 minutes per 10M steps. This allowed for extensive hyperparameter tuning with Optuna within a highly reasonable timeframe.

3.5.2 Hyperparameters Optimization

We conducted hyperparameter optimization using **Optuna** over 50 trials, with a total runtime of approximately 16 hours. The hyperparameter search used the Tree-structured Parzen Estimator (TPE) to guide sampling toward promising regions of the search space by modeling the performance of good and bad configurations, while Hyperband was employed to efficiently allocate resources and prune underperforming trials through early stopping. Full details are reported in Appendix A.

As shown in Figure 3, MLP models with featurized environment encoding consistently outperformed CNNs with lossless environment encoding, converging faster and achieving higher final returns.

The parallel coordinates plots (Figure 4) show that stable and high-performing configurations concentrate in a narrow region of the hyperparameter space: an entropy coefficient of 0.01–0.05, GAE λ between 0.9 and 0.95, 800-step rollout segments, and learning rates of 10^{-4} – 10^{-3} . The MLP uses a 256-unit hidden layer. Sharing parameters between agents consistently improved performance, whereas sharing a backbone between the policy and value networks did not.

The hyperparameters reported in Table 2 were used to train both single-layout and multi-layout agent pairs. Figure 2 shows the episode rewards over training steps for the multi-layout agent pair across the different layouts.

3.5.3 Results

Table 1: Total reward achieved by agent pairs across training and holdout layouts, averaged over 10 evaluation episodes.

	Training Layouts					Holdout Layouts	
	Cramped Room	Asymm. Adv.	Coord. Ring	Forced Coord.	Counter Circuit	Cramped Tomato	5x5
Trained on all training layouts	220	<u>460</u>	<u>260</u>	260	-	<u>315</u>	-
Trained on Cramped Room	<u>240</u>	-	-	-	-	-	-
Trained on Asymm. Adv.	-	<u>460</u>	-	-	-	-	-
Trained on Coord. Ring	-	-	200	-	-	-	-
Trained on Forced Coord.	-	-	-	<u>300</u>	-	-	-
Trained on Counter Circ.	-	-	-	-	-	-	-

The results reported in Table 1 demonstrate that IPPO achieves robust performance. Notably, both the single-layout specialized pairs and the multi-layout agents converge to high-reward policies on the majority of training environments, successfully solving *Cramped Room*, *Asymmetric Advantage*, *Coordination Ring*, and *Forced Coordination*.

However, *Counter Circuit* remains unsolved, yielding zero reward for all trained agent pairs. This failure appears to result from a specific inability to adapt to ingredient changes; *Counter Circuit* is the only layout requiring the delivery of tomato-based dishes. The agents, however, persist in behaviors related to onion-based recipes, failing to adjust to the tomato availability.

Single-layout agent pairs show no form of generalization and fail completely on new layouts. Even if the multi-layout agent pair has some form of generalization and successfully solves the *Cramped Tomato* layout, it still persists with onion-based deliveries. Moreover, it struggles in more complex environments with layouts never seen before.

4 Discussion & Conclusion

Overall, the results indicate the emergence of non-trivial generalization behavior. Training on a distribution of layouts enables agents to learn more flexible policies that transfer to unseen environments, as evidenced by performance on *Cramped Tomato*. However, this generalization remains limited and degrades rapidly when agents are evaluated on layouts slightly more complex than those encountered during training.

The failure on *Counter Circuit* highlights the influence of layout-specific biases. This layout requires exclusively tomato-based deliveries, whereas the learned policy remains biased toward onion-based deliveries. We hypothesize that this stems from the fact that four of the five training layouts involve onion-based deliveries (see Figure 1). Such over-representation likely biases the policy, trapping agents in suboptimal behaviors, effectively “overfitting” to the training distribution. To achieve stronger generalization, it appears crucial to extend training with randomized or procedurally generated layouts to ensure a more uniform distribution of ingredients and map structures. Since Overcooked-AI supports the integration of new layouts, future work could explore whether increased diversity in training environments improves transfer performance.

In conclusion, these results show that multi-layout training can improve policy generalization, and IPPO proves highly effective even in multi-agent settings. Nonetheless, generalization to unseen scenarios remains challenging, motivating future work on more diverse training environments, refined reward shaping, and other strategies that enhance adaptability to novel layouts.

References

- [1] Zikai Cai, Muchen Ge, Zilong Guo, Yufen Huang, An Liu, Huating Wen, and Shengxiang Xu. Deep learning based overcooked play agent, 2021. URL <https://overcookedcooker.wordpress.com/paper/>.
- [2] Micah Carroll, Rohin Shah, Mark K. Ho, Thomas L. Griffiths, Sanjit A. Seshia, Pieter Abbeel, and Anca Dragan. On the utility of learning about humans for human-ai coordination, 2020. URL <https://arxiv.org/abs/1910.05789>.
- [3] Christian Schroeder de Witt, Tarun Gupta, Denys Makoviichuk, Viktor Makovychuk, Philip H. S. Torr, Mingfei Sun, and Shimon Whiteson. Is independent learning all you need in the starcraft multi-agent challenge?, 2020. URL <https://arxiv.org/abs/2011.09533>.
- [4] Shengyi Huang, Rousslan Fernand Julien Dossa, Antonin Raffin, Anssi Kanervisto, and Weixun Wang. The 37 implementation details of proximal policy optimization. *ICLR Blog Post*, 2022. <https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/>.
- [5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, February 2015. ISSN 00280836. URL <http://dx.doi.org/10.1038/nature14236>.
- [6] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative, multi-agent games, 2022. URL <https://arxiv.org/abs/2103.01955>.

A Hyperparameter Optimization Results

Table 2: Final hyperparameter configuration determined by Optuna.

Hyperparameter	Type	Search Space	Best Value
Network Type	Categorical	{MLP, CNN}	MLP
Env Encoding	Categorical	{Featurized, Lossless}	Featurized
Shared Backbone	Categorical	{True, False}	False
MLP Hidden Dim	Categorical	{64, 128, 256}	256
Learning Rate	Log-float	$[10^{-5}, 10^{-3}]$	4.5×10^{-4}
Entropy Coef	Log-float	$[0.01, 0.3]$	0.0135
GAE Lambda	Float	$[0.9, 0.99]$	0.94
Update Epochs	Integer	$[4, 10]$	10
Num Steps	Categorical	{400, 800}	800
Parameter Sharing	Categorical	{True, False}	True



Figure 3: **Optimization History**: Evolution of objective values across 50 trials.

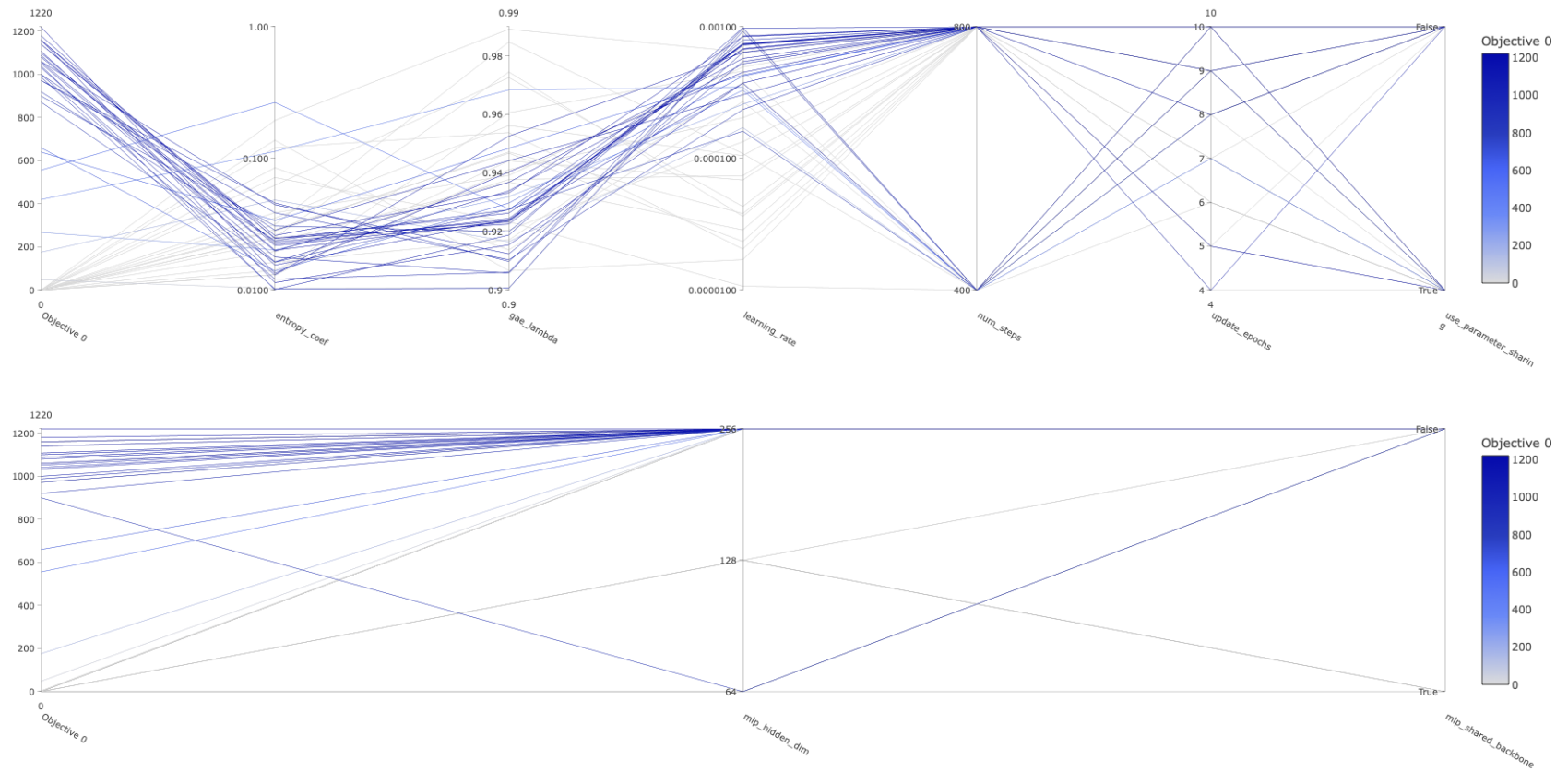


Figure 4: **Parallel Coordinate Plots** illustrating the relationship between hyperparameters and total reward. The concentration of high-performing trials (dark blue) highlights optimal regions for critical hyperparameters.